

## The need for DevOps in IT

We have seen how DevOps helps in reducing the friction between the development and operations teams. Now let's see what effects DevOps has if it's integrated into our software development process.

- Bugs can be identified and resolved at an early stage of development due to better communication and collaboration between teams.
- In the Agile model, with DevOps, there is better management of resources since there is less free time due to the presence of cross-functional teams.
- One striking feature of DevOps is the use of version control systems that can reduce the time and effort of the coder.
- Implementation of DevOps also provides an opportunity to dedicate more time and effort to innovation and research.

## Behaviour driven development (BDD)

We have just discussed the Agile model and DevOps, and the need to implement these in today's software development scenario. But since DevOps involves various teams such as developers, testers, stakeholders, etc., sometimes there can be a lack of communication between them too. Developers can misinterpret the needs of business stakeholders, and testers can misunderstand the ideas of developers. This can cause a huge negative impact on the overall productivity of the team, affecting actual deliverables. So there is a need for a common language to drive the team and bridge the communication gap. In addition, the following disadvantages were observed in Agile projects:

- Usually, user stories in the Agile model are more focused on the users and their needs rather than on the business logic of the feature. This aspect gets overlooked during sprint planning meetings and can lead to the development of unnecessary features.
- Acceptance criteria, which indicate the completion of a user story, can be understood differently by different individuals.
- Most often, Agile teams adopt the test driven development (TDD) approach, but this approach is very costly.

These weaknesses in the Agile model led to the birth of behaviour driven development (BDD). BDD is also an Agile software development process, which encourages effective communication between the business team, developers, project managers and QA by increasing the focus on business goals and business values. It was conceived by Dan North in 2003 who defines it as follows: "BDD is a second-generation, outside-in, pull-based, multiple-stakeholder, multiple-scale, high-automation, Agile methodology. It describes a cycle of interactions with well-defined outputs, resulting in the delivery of working, tested software that matters."

BDD is an extension of TDD with a few minor differences such as:

1. Tests in BDD are written in plain English.
2. Tests are more behaviour focused and deal with the functionality of the application.
3. BDD uses extensive examples.
  - BDD has a lot of advantages over the traditional TDD approach. A few of these have been listed below.
  - Since BDD involves plain English, it encourages collaboration among different parties involved in the software development cycle. Everyone has a clear understanding of the project and can contribute to planning sessions constructively.
  - BDD puts more emphasis on business values and needs, which can help developers in delivering better results because they can understand what the stakeholder wants and work accordingly.
  - Due to a single language being used, there is less scope for misunderstanding and misconceptions.
  - BDD also leads to better acceptance testing, since the user can also understand the user stories.

## BDD testing tools

There are various tools available in the market, which support the BDD framework. Some of these are Cucumber, Specflow, Behave, JBehave, JBehave Web, Lettuce, Behat, Kahlan, etc.

**Cucumber:** Cucumber is the most widely used open source tool that supports behaviour driven development. It allows you to write application behaviour in a simple, English-like language known as Gherkin. It is written in Ruby but can support various languages like Java, JS, Python, .NET, C++, etc. Due to its wide language support, it can integrate with almost all testing tools and frameworks. You can read more about this tool at <https://cucumber.io/docs>.

**Gherkin:** Gherkin is a language that Cucumber understands. It is defined on GitHub as, "... a business readable, domain-specific language that lets you describe the software's behaviour without detailing how that behaviour is implemented." Gherkin is easy to learn and understand by non-programmers, but allows illustration of business rules in real-world domains. It has a particular syntax that contains keywords such as *scenario*, *given*, *then*, *and*, *examples*, *but*, etc. A sample Gherkin document looks like what's shown below:

```
Feature: Refund item
Scenario: Jeff returns a faulty microwave
Given Jeff has bought a microwave for $100
And he has a receipt
When he returns the microwave
Then Jeff should be refunded $100
```

So you can see that we have specified the behaviour of the 'Refund item' system using various keywords underlined above in plain English text. You can study Gherkin and its various rules at <https://cucumber.io/docs/reference>.